



Aplicações Modulares. Serviços



Módulos

Módulos são uma novidade do JDK 9. Podem ajudar as grandes aplicações a reduzir a complexidade da sua gestão e pequenas aplicações deixando o programador especificar quais módulos pretende usar, reduzindo a sua pegada, tornando-se mais eficiente. Isto importa sobretudo para dispositivos com poucos recursos como dispositivos IoT (Internet of Things).

O suporte para módulos é fornecido tanto por elementos da linguagem, como várias keywords, bem como melhorias ao javac, java e outras ferramentas do JDK. Além disso, foram introduzidas novas ferramentas e formatos de ficheiros.

Um módulo é um agrupamento de packages e recursos que podem ser utilizados através do nome do módulo. A declaração de um módulo especifica o seu nome e define a relação que um módulo e os seus pacotes têm com outros módulos. As declarações de módulos são statements num source file de Java e são suportadas por várias keywords, abaixo. De salientar que estas apenas funcionam na declaração de módulo, em outros casos serão lidas como parâmetros. Por serem sensíveis ao contexto, chamam-se restricted keywords.

- exports
- provides
- uses
- module
- requires
- with
- open
- to
- opens
- transitive



Módulos

A declaração de um módulo é feita no ficheiro `module-info.java`. Sendo normalmente compilado, este ficheiro é conhecido por `module descriptor`. O `module descriptor` apenas pode conter a definição de um módulo, não podendo integrar outros tipos de declarações. A declaração de um módulo começa obrigatoriamente com a keyword `module`:

```
module moduleName {  
    // module definition  
}
```

O nome do módulo deve ser um identificador de Java válido ou uma sequência de identificadores separados por pontos. A definição do módulo é limitada pelas chavetas. Pode estar vazia, dando apenas nome ao módulo, mas normalmente terá uma ou mais cláusulas que o definem.



Um módulo simples

Começamos por definir se este nosso módulo necessita (depende) de outro. Esta necessidade é explícita através da keyword **requires**. Por defeito, a presença de um módulo requerido por outro é verificada tanto no tempo de compilação como quando corremos o programa. De seguida, podemos definir se e quais dos packages deste módulo poderão estar acessíveis a outros módulos. Fazemos isso com a keyword **exports**. Os parâmetros e métodos públicos e protegidos de um pacote estão acessíveis a outros módulos apenas se forem explicitamente exportados.

1. Criamos um diretório chamado **myapp**. Esta será a raiz da nossa aplicação.
2. Dentro dessa pasta criar outra chamada **appsrc**. Esta é a raiz para o código da nossa aplicação.
3. Dentro dessa pasta, criar outra intitulada **appstart**, e por sua vez outra dentro desta, também intitulada **appstart**. Nesta última pasta, criar uma nova chamada **myappdemo**. Começando na **appsrc** e terminando na pasta mais profunda, deveremos ter algo assim: `appsrc/appstart/appstart/myappdemo`.
4. Também dentro de **appsrc**, criar **appfuncs**. Dentro desta, criar outra também chamada **appfuncs**. Por sua vez, esta terá uma pasta chamada **simplefuncs**. Começando em **appsrc**, devemos ter a seguinte estrutura: `appsrc/appfuncs/appfuncs/simplefuncs`.

Depois disto, podemos criar o código fonte. Este nosso exemplo usa 4 source files. 2 definem a aplicação. O primeiro é **SimpleMathFuncs.java**.



simpleMathFuncs.java

```
// Some simple math functions.

package appfuncs.simplefuncs;

public class SimpleMathFuncs {

    // Determine if a is a factor of b.
    public static boolean isFactor(int a, int b) {
        if((b%a) == 0) return true;
        return false;
    }

    // Return the smallest positive factor that a and b have in common.
    public static int lcf(int a, int b) {
        // Factor using positive values.
        a = Math.abs(a);
        b = Math.abs(b);

        int min = a < b ? a : b;

        for(int i = 2; i <= min/2; i++) {
            if(isFactor(i, a) && isFactor(i, b))
                return i;
        }
    }
}
```

```
        return 1;
    }

    // Return the largest positive factor that a and b have in common.
    public static int gcf(int a, int b) {
        // Factor using positive values.
        a = Math.abs(a);
        b = Math.abs(b);

        int min = a < b ? a : b;

        for(int i = min/2; i >= 2; i--) {
            if(isFactor(i, a) && isFactor(i, b))
                return i;
        }

        return 1;
    }
}
```



myAppDemo.java

```
// Demonstrate a simple module-based application.
package appstart.mymodappdemo;

import appfuncs.simplefuncs.SimpleMathFuncs;

public class MyModAppDemo {
    public static void main(String[] args) {

        if(SimpleMathFuncs.isFactor(2, 10))
            System.out.println("2 is a factor of 10");

        System.out.println("Smallest factor common to both 35 and 105 is " +
            SimpleMathFuncs.lcf(35, 105));

        System.out.println("Largest factor common to both 35 and 105 is " +
            SimpleMathFuncs.gcf(35, 105));

    }
}
```



Definições dos módulos

```
// Module definition for the functions module.  
module appfuncs {  
  // Exports the package appfuncs.simplefuncs.  
  exports appfuncs.simplefuncs;  
}
```

```
// Module definition for the main application module.  
module appstart {  
  // Requires the module appfuncs.  
  requires appfuncs;  
}
```



Definições dos módulos

```
// Module definition for the functions module.  
module appfuncs {  
  // Exports the package appfuncs.simplefuncs.  
  exports appfuncs.simplefuncs;  
}
```

```
// Module definition for the main application module.  
module appstart {  
  // Requires the module appfuncs.  
  requires appfuncs;  
}
```




Java.base

Com a introdução do JDK 9, a própria biblioteca de Java foi toda movida para módulos. Mas então como é que conseguimos usar `System.out.println()` no exemplo anterior sem incluir o respetivo módulo?

A resposta está no `Java.base`. Todos os módulos, de forma automática, vão importar este. Tendo em conta que inclui pacotes essenciais como `java.lang`, `java.io` ou `java.util`, decidiram fazê-lo assim para que não tivesse que se fazer o `requires` a este módulo em todos os programas Java.



Questões

Com a introdução do JDK 9, a própria biblioteca de Java foi toda movida para módulos. Mas então como é que conseguimos usar `System.out.println()` no exemplo anterior sem incluir o respetivo módulo?

A resposta está no `Java.base`. Todos os módulos, de forma automática, vão importar este. Tendo em conta que inclui pacotes essenciais como `java.lang`, `java.io` ou `java.util`, decidiram fazê-lo assim para que não tivesse que se fazer o `requires` a este módulo em todos os programas Java.

E o código antigo, que não usa módulos, como funciona?

Todo o código que não faça parte de um módulo fica a fazer parte de um módulo sem nome. Este tem 2 atributos importantes. Por um lado, todas as `packages` são exportadas automaticamente para que possam estar disponíveis. Por outro lado, este `package` pode aceder a qualquer outro módulo para garantir a compatibilidade do código antigo.

Outra parte importante é o uso automático do caminho da classe em vez do caminho do módulo. Ao compilar programa sem módulos, o mecanismo de caminho da classe é utilizado como aconteceu desde o início do Java até recentemente. Como resultado, o programa é compilado e executado como anteriormente.



exports e requires

É possível exportar um package para um módulo específico. Por segurança, podemos precisar desta feature. Para o fazermos, podemos utilizar o exports da seguinte maneira:

```
exports nomeDoPackage to nomeDoModulo1, nomeDoModulo2, nomeDoModulo3;
```

Por outro lado, se tivermos 3 módulos A, B e C em que A precisa de B e B precisa de C, tudo funcionará corretamente se A não precisar de C. No entanto, caso necessite, terá de importar C. Se B for usado por muitos módulos, estar a importar C em todos eles é uma tarefa complicada. Dessa forma, é possível alterar a declaração do requisito de C no módulo B através da keyword **transitive**.

```
// A's module-info file:
module A {
    requires B;
}

// B's module-info file.
module B {
    exports somepack;
    requires C;
}
```

```
// B's module-info file.
module B {
    exports somepack;
    requires transitive C;
}
```



Serviços

Muitas vezes em programação convém separar o que deve ser feito de como deve ser feito. Para tal, podemos utilizar interfaces. A interface especifica “o quê” enquanto a classe que a implementa especifica o “como”. Este conceito pode ser expandido para que a classe que implementa seja fornecida por código fora do programa, através do uso de um “plug-in”. Com esta abordagem, as capacidades de uma aplicação podem ser melhoradas ou alteradas mudando apenas o plug-in. O núcleo da aplicação mantém-se. Uma forma de o Java suportar uma arquitectura modular é através de serviços e disponibilizadores de serviços. Devido à sua importância, especialmente em aplicações comerciais de tamanho acrescido, o sistema de módulos de Java dá-lhes suporte.

Antes de começarmos a falar sobre os serviços, devemos salientar que as aplicações que usam serviços e disponibilizadores de serviços são tipicamente bastante sofisticadas. Tendo isso em conta, esta feature poderá não ser utilizada muitas vezes. Vamos falar deles por serem uma parte importante do sistema de módulos e convém que saibamos como funciona.



Serviços e disponibilizadores de serviços

Em Java, um serviço é uma unidade de um programa cuja funcionalidade é definida por uma interface ou classe abstrata. A sua implementação é fornecida por um service provider.

Um serviço pode ser, por exemplo, um sistema de tradução de uma língua para outra. Neste caso, o serviço suporta tradução de uma forma geral. O service provider fornece uma tradução específica, como tradução português-inglês ou alemão-francês. Como todos os disponibilizadores de serviços implementam a mesma interface, diferentes tradutores podem ser usados para traduzir diferentes línguas sem ter de mudar o core da aplicação.

Service providers são suportados pela classe ServiceLoader. Esta é uma classe genérica no módulo java.util.

```
class ServiceLoader<S>
```

S especifica o tipo do serviço. Service providers são carregados pelo método load().

```
public static <S> ServiceLoader<S> load(Class <S> serviceType)
```

ServiceType especifica o objeto Classe para o tipo de serviço desejado.



Keywords de serviços

Módulos suportam serviços através do uso das keywords **provides**, **uses** e **with**. Essencialmente, um módulo especifica que fornece um serviço com **provides**. Um módulo indica que precisa de um serviço com **uses** e o tipo específico do service provider é declarado com **with**. Quando usados em conjunto, permitem ao programador especificar que um módulo fornece um serviço, um módulo que precisa dele e a implementação específica desse mesmo serviço.

```
provides serviceType with implementationTypes;
```

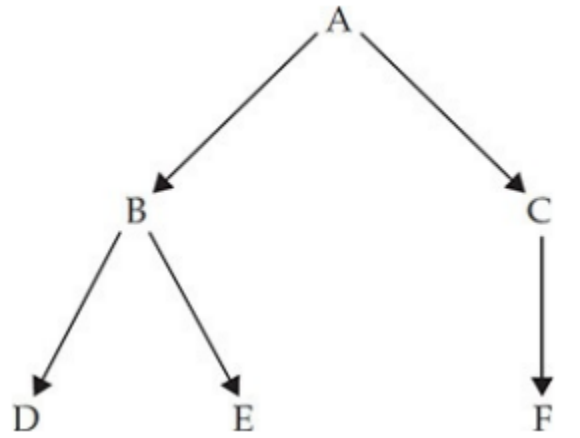
serviceType especifica o tipo do serviço, maioritariamente uma interface, apesar de classes abstratas também serem usadas.

```
uses serviceType;
```



Grafos de módulos

Durante a compilação, o compilador resolve as relações de dependência entre módulos criando um grafo de módulos que as representa. Este processo garante que todas as dependências são resolvidas, incluindo as indiretas.





Open modules

Existem mais 3 features de módulos que poderão ser úteis: **open**, **opens** e **requires static**. Cada uma destas é desenhada para suportar uma situação específica e são todas aspectos avançados do sistema de módulos.

Como já vimos, os tipos nos packages de um module apenas estão acessíveis se forem exportadas através do comando `exports`. No entanto, poderá haver situações em que é útil permitir acesso em tempo de execução a todos os packages do módulo, seja exportado ou não. Para permitir isso, podemos criar um open module. Este é declarado antecedendo module de open na sua declaração.

```
open module moduleName {  
  // module definition  
}
```

Graças a esta declaração, todos os tipos de todos os packages de um módulo ficarão disponíveis durante a execução. No entanto, apenas os pacotes explicitamente exportados estarão disponíveis em tempo de compilação.



Open modules

O comando **opens** permite que um módulo abra um package específico para acesso em tempo de execução por outros módulos e para acesso refletivo em vez de abrir o módulo por completo. Para tal, usamos o **opens** da seguinte forma.

```
opens packageName;
```

É possível usar a keyword **to** para especificar quais os módulos aos quais o package ficará aberto. Também é importante perceber que **opens** não garante acesso em tempo de compilação.



requires static

Como sabemos, a keyword **requires** indica uma dependência que é cumprida em tempo de compilação e execução. No entanto, podemos não necessitar da mesma em tempo de execução. É para isso que o **requires static** serve. Apenas usa o **requires** em tempo de compilação. Neste caso, o package em questão torna-se opcional em tempo de execução.